

Mini Project 3

EECE 7398 Legged Robotics

Code Listing

func_zero_dynamics.m

```
1 % Simulation zero dynamics of 3 link biped in the sagittal plane
2 %
3 % Inputs:
4 %     t: time
5 %     z: cyclic variables [q1, dq1]
6 %     a: bezier coefficient for q2 - a(1:5), and q3 - a(6:10)
7 %     s_params: [z_min, z_max] - max and min angles in gait
8 %
9 % Outputs:
10 %     dz = [dq1, ddq1]
11 %
12 function dz = func_zero_dynamics(t,z,a,s_params)
13
14 z_min = s_params(1);
15 z_max = s_params(2);
16 delq = z_max - z_min;
17
18 q1 = z(1); dq1 = z(2);
19
20 % Computes gait timing variable s
21 %
22 % Inputs:
23 %     q1: cyclic variable
24 %     q1_min: minimum value for q1 during gait
25 %     q1_max: max value for gait
26 %
27 % Output:
28 %     s: gait timing variable
29 %
30 s = func_gait_timing(q1,z_min,z_max);
31 if s > 1; s = 1;
32 elseif s < 0; s = 0;
33 end
34
35 alpha2 = a(1:5);
36 alpha3 = a(6:10);
37
38 % find based on s and bezier definition
39 q2 = bezier(s, 4, alpha2);
40 q3 = bezier(s, 4, alpha3);
41 db_ds2 = d_ds_bezier(s, 4, alpha2);
42 db_ds3 = d_ds_bezier(s, 4, alpha3);
43
44 dq2 = db_ds2 * (dq1/delq);
45 dq3 = db_ds3 * (dq1/delq);
46
47 q = [q1, q2, q3];
48 dq = [dq1, dq2, dq3];
49
50 % Get model parameters
51 [r,m,Mh,Mt,l,g] = func_model_params;
```

```

52 params = [r,m,Mh,Mt,l,g];
53
54 % Compute D,C,G,B matrices
55 % Inputs:
56 %     q
57 %     dq
58 %     params
59 %
60 % Output: [D,C,G,B]
61 %
62 [D,C,G,~] = func_compute_D_C_G_B(q,dq,params);
63
64
65 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Apply dynamics partitioning to compute the zero dynamics
66 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% equations %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
67
68 % E.g.,
69 D1 = D(1, 1);
70 D2 = D(1, 2:3);
71 H1 = C(1,:)*dq' + G(1);
72
73 %------%
74
75 % Get beta1 term to compute dz
76 %
77 % Inputs: s, [dq1, delq], [alpha2, alpha3]
78 %     s: gait timing variable
79 %     dq1: velocity of cyclic variable = z2
80 %     delq: difference between z_max and z_min in s, needed when
81 %           computing db/ds
82 %     alpha2: Bezier coefficient (1st to 5th) for q2
83 %     alpha3: Bezier coefficient (1st to 5th) for q3
84 %
85 % Output:
86 %     beta1: one of 2 matrices obtained when computing ddq_b (acceleration
87 %           of body coordinates)
88 %     y = h(x) = q_b - b(s), when y = 0, q_b = b(s)
89 %           taking two time derivatives we get:
90 %     ddq_b = d/ds(db(s)/ds*ds/dt)*ds/dt + db(s)/ds * d^2(s)/dt^2
91 %           = beta1 + beta2*ddq_1
92 %           Note: d^2(s)/dt^2 = 1/delq*d^2(q1)/dt^2
93 %
94 %     beta1 = d/ds(db(s)/ds*ds/dt)*ds/dt
95 %     beta2 = db(s)/ds*1/delq
96 %
97 beta1 = func_compute_beta1(s, [dq1, delq], [alpha2, alpha3]);
98
99 % Compute 1st partial derivative of Bezier polynomial
100 % Inputs:
101 %     s: gait timing variable
102 %     M: order of Bezier polynomial, in this case 4th order
103 %     alpha: Coefficients for the polynomial, need M+1
104 %
105 % Outputs:
106 %     db/ds
107 %
108
109 beta2 = [db_ds2; db_ds3]/delq;
110
111 %------%
112
113 dz(1) = z(2);
114 dz(2) = (-H1 - D2*beta1) / (D1 + D2*beta2);
115
116 dz = [dz(1), dz(2)]';
117 end

```

Optimize.m

```

1 % Optimize intial conditions for ZD states and Bezier coeffs using fmincon
2 %
3 % Zero dynamics simulator takes pre-impact condition so that should be I.C.
4 % for fmincon as well
5
6 %-----%
7
8 %[q1, dq1] pre impact conditions - last known working solution (old Bezier)
9 z_minus = [-0.3407, -2.6352];
10
11 % Bezier coefficients - last known working solution
12 alpha = [0.4553, 0.6698, 0.8463]; %for q2 - alpha 3rd - 5th
13 gamma = [2.4988, 2.6419, 2.8039]; %for q3 - alpha 3rd - 5th
14
15
16 % f0 = [q10, dq10, alpha(3-5)_q2, alpha(3-5)_q3]
17 % q10: pre-impact initial angle for q1
18 % dq10: pre-impact initial velocity for dq1
19 % alpha(3-5)_q2:
20 % 3rd to 5th Bezier coefficient for q2
21 % alpha(3-5)_q3:
22 % 3rd to 5th Bezier coefficient for q3
23 %
24 f0 = [z_minus, alpha, gamma]; %parameters that need to be optimized
25
26 %-----%
27
28 % Lower and upper bounds for optimizer - [q1 q1_dot, alpha q2, alpha q3]
29 %
30 % epsilon for angles
31 eps = 0.15;
32 % epsilon for velocity
33 eps_v = 0.5;
34 lb = [z_minus(1)-eps, z_minus(2)-eps_v, alpha-eps, gamma-eps];
35 ub = [z_minus(1)+eps, z_minus(2)+eps_v, alpha+eps, gamma+eps];
36
37 %-----%
38
39 %%% Run optimizer
40
41 opts = optimoptions('fmincon','Algorithm','interior-point',...
42                     'MaxIterations',1000,...
43                     'StepTolerance',1e-6,...
44                     'FunctionTolerance',1e-2);
45 opts.Display = 'iter';
46 opts.EnableFeasibilityMode = true;
47 opts.SubproblemAlgorithm = 'cg';
48
49
50 % using only bounds and nonlinear constraints
51 %
52 % Outputs: f - optimized parameters [qt, dqt, qt_LR_max, alphaR, alphaL]
53 % J - Cost to minimize
54 %
55 [f, J] = fmincon(@(f) pass_cost_ZD(f), f0, [],[],[],[], lb,ub, @confuneq,opts);
56
57 disp('solution found!');
58
59 f
60
61 % ---- PRINT: final solution summary ----
62 fprintf('\n== Final Solution ==\n');
63 fprintf('f = [%4f, %4f, %4f, %4f, %4f, %4f, %4f, %4f]\n', f);
64 fprintf('Cost J = %.6f\n', J);
65 alpha2_full = [-f(5), -f(4), f(3:5)];
66 [r,~,~,~,~] = func_model_params;

```

```

67 q1_preimpact = f(1); q2_preimpact = alpha2_full(5);
68 step_length = abs(r*sin(q1_preimpact) + r*sin(q1_preimpact - q2_preimpact));
69 fprintf('Step length = %.4f m\n', step_length);
70 % ---- END PRINT ----
71
72 % Function only exists to pass cost as a scalar output for fmincon, actual
73 % cost calculation happens in func_cost_ZD
74 %
75 function J = pass_cost_ZD(y)
76
77 [J, ~] = func_cost_ZD(y);
78
79 end
80
81 %------%
82
83 % Simulate a single step of zero dynamics and get cost
84 %
85 % Inputs: [z_plus, a, s_params]
86 %         z_plus: [q1, dq1] post impact condition
87 %         a: [alpha2, alpha3] Bezier coefficients for q2 and q3
88 %         s_params: [q1_min, q1_max]
89 %
90 % Outputs: [J, z_sol]
91 %          J: Cost
92 %          z_sol: solution to zero dynamics ODE45
93 %
94 function [J, z_sol] = func_cost_ZD(f)
95
96 %%%% Simulate
97
98 % Simulation of a single step of ZD to using preimpact conditions
99 %   applies impact first then simulates zero dynamics
100 %
101 % Inputs: f = [q10, dq10, alpha(3-5)_q2, alpha(3-5)_q3]
102 %          q10: pre-impact initial angle for q1
103 %          dq10: pre-impact initial velocity for dq1
104 %          alpha(3-5)_q2:
105 %                      3rd to 5th Bezier coefficient for q2
106 %          alpha(3-5)_q3:
107 %                      3rd to 5th Bezier coefficient for q3
108 %
109 % Outputs:
110 %          t_sol - time (s) of zero dynamics
111 %          z_sol - [q1, dq1] post impact dynamics
112 %
113 [t_sol, z_sol] = sim_zero_dynamics(f);
114
115 %%%% Compute cost
116
117 % Variables needed to compute control action
118 % Match sim_zero_dynamics convention: q1_min=-f(1)=+0.3065 (post-impact,
119 % start of swing), q1_max=f(1)=-0.3065 (pre-impact, end of swing).
120 % This ensures s = (q1 - q1_min)/(q1_max - q1_min) runs 0->1 through z_sol.
121 q1_min = -f(1);
122 q1_max = f(1);
123
124 s_params = [q1_min, q1_max];
125
126 alpha2 = [-f(5), -f(4), f(3:5)];
127 alpha3 = [f(8)-f(5), -f(7)+2*f(6), f(6:8)];
128
129 a = [alpha2, alpha3];
130
131 J = 0;
132 for i = 1:length(t_sol)
133     % Post compute control action
134     %

```

```

135 % Inputs: [z, a, s_params]
136 %       z: [q1, dq1]
137 %       a: [alpha2, alpha3] - 1st to 5th Bezier coefficient for q2 and q3
138 %       s_params: [q1_min, q1_max]
139 %
140 % Output:
141 %       u: control action
142 %
143 u = func_compute_control_action(z_sol(i,:),a,s_params);
144
145 % Cost = sum of norms of control action
146 J = J + norm(u)^2;
147 end
148 % Normalize after loop
149 J = J / length(t_sol);
150
151
152 end
153
154 %-----%
155
156 % Nonlinear constraint for optimizer
157 %
158 % Inputs: f: [q1, dq1, alpha2(3:5), alpha3(3:5)]
159 %
160 % Output: [c, ceq]
161 %       c: nonlinear inequality constraint
162 %       ceq: nonlinear equality constraint
163 %
164 function [c,ceq] = confuneq(f)
165
166 z_i = f(1:2); %Pre-impact states
167
168 % Simulation of a single step of ZD to using preimpact conditions
169 % applies impact first then simulates zero dynamics
170 [~, z_sol] = sim_zero_dynamics(f);
171
172 z_f = z_sol(end,:); %final values after swing phase and right before the next impact
173
174 % Reconstruct full 5-coeff Bezier from optimized [alpha(3),alpha(4),alpha(5)]
175 % Convention: alpha2(1)=-alpha2(5), alpha3(1)=alpha3(5)-alpha2(5) (leg relabeling geometry)
176 alpha2_full = [-f(5), -f(4), f(3), f(4), f(5)];
177 alpha3_full = [f(8)-f(5), -f(7)+2*f(6), f(6), f(7), f(8)];
178
179 % Nonlinear inequality constraints
180 c = [];
181 % Nonlinear equality constraints: periodicity only
182 ceq = [z_i(1) - z_f(1); % q1 periodicity
        z_i(2) - z_f(2)]; % dq1 periodicity
183
184
185 % ---- PRINT: ceq debug ----
186 fprintf('[ceq] z_i=[%.4f, %.4f] z_f=[%.4f, %.4f] ceq=[%.2e, %.2e]\n', ...
        z_i(1), z_i(2), z_f(1), z_f(2), ceq(1), ceq(2));
187
188 [r,~,~,~,~] = func_model_params;
189 q1_preimpact = f(1); q2_preimpact = alpha2_full(5);
190 step_length = abs(r*sin(q1_preimpact) + r*sin(q1_preimpact - q2_preimpact));
191 fprintf('[step] step_length=%.4f m\n', step_length);
192 % ---- END PRINT ----
193
194 end

```

sim_and_plot_ZD.m

```

1
2 % Include util and autogen folders
3 set_path

```

```

4
5 % f = [q10, dq10, alpha(3-5)_q2, alpha(3-5)_q3]
6 % q10: pre-impact initial angle for q1
7 % dq10: pre-impact initial velocity for dq1
8 % alpha(3-5)_q2:
9 % 3rd to 5th Bezier coefficient for q2
10 % alpha(3-5)_q3:
11 % 3rd to 5th Bezier coefficient for q3
12 %
13 % Manually tuned (working):
14 f = [-0.4907, -2.3947, 0.3053, 0.6256, 0.8814, 2.6360, 2.7919, 2.9537];
15
16 z_tot = [];
17
18 n = 5;
19
20 for i = 1:n
21
22 % Simulation of a single step of ZD to using preimpact conditions
23 % applies impact first then simulates zero dynamics
24 %
25 % Inputs: f = [q10, dq10, alpha(3-5)_q2, alpha(3-5)_q3]
26 % q10: pre-impact initial angle for q1
27 % dq10: pre-impact initial velocity for dq1
28 % alpha(3-5)_q2:
29 % 3rd to 5th Bezier coefficient for q2
30 % alpha(3-5)_q3:
31 % 3rd to 5th Bezier coefficient for q3
32 %
33 % Outputs:
34 % t_sol - time (s) of zero dynamics
35 % z_sol - [q1, dq1] post impact dynamics
36 %
37 [t_sol, z_sol] = sim_zero_dynamics(f);
38
39 f(1:2) = z_sol(end,:);
40
41 z_tot = [z_tot; z_sol];
42
43 end
44
45 figure
46 plot(z_tot(1,1),z_tot(1,2),'x'), hold on
47 plot(z_tot(:,1),z_tot(:,2))
48 hold off, grid on
49 title('Phase portrait of q_1 vd dq_1')
50 xlabel('q_1 (rads)')
51 ylabel('dq_1 (rads/s)')
52 legend('Start','Trajectory')

```

Outputs

Mini_Project3_CW (command window output)

	Iter	F-count	f(x)	Feasibility	First-order optimality	Norm of step
	19	207	3.388795e+02	5.087e-10	1.611e+02	6.284e-05
[ceq]	z_i=[-0.4907, -2.3946]		z_f=[-0.4907, -2.3946]		ceq=[4.00e-15, 5.80e-10]	
[step]	step_length=0.8520 m					
	20	216	3.388792e+02	5.805e-10	3.109e+02	8.268e-05
[ceq]	z_i=[-0.4907, -2.3947]		z_f=[-0.4907, -2.3947]		ceq=[3.66e-15, 1.02e-05]	
[step]	step_length=0.8520 m					
	21	227	3.388790e+02	1.453e-11	3.853e+02	5.002e-05
[ceq]	z_i=[-0.4907, -2.3946]		z_f=[-0.4907, -2.3946]		ceq=[4.94e-15, 2.53e-12]	
[step]	step_length=0.8520 m					

```

12      22      236      3.388789e+02      2.530e-12      2.908e+02      2.900e-06
13 [ceq] z_i=[-0.4907, -2.3946] z_f=[-0.4907, -2.3946] ceq=[3.77e-15, 1.01e-05]
14 [step] step_length=0.8520 m
15      23      252      3.388789e+02      1.247e-11      3.666e+02      6.523e-06
16
17 Local minimum possible. Constraints satisfied.
18
19 fmincon stopped because the size of the current step is less than
20 the value of the step size tolerance and constraints are
21 satisfied to within the value of the constraint tolerance.
22
23 solution found!
24
25 f =
26
27 Columns 1 through 7
28
29      -0.4907      -2.3947      0.3053      0.6256      0.8814      2.6360      2.7919
30
31 Column 8
32
33      2.9537
34
35
36 === Final Solution ===
37 f = [-0.4907, -2.3947, 0.3053, 0.6256, 0.8814, 2.6360, 2.7919, 2.9537]
38 Cost J_avg = 338.8789
39 Step length = 0.8520 m

```

Zero Dynamics Simulation (sim_and_plot_ZD.m)

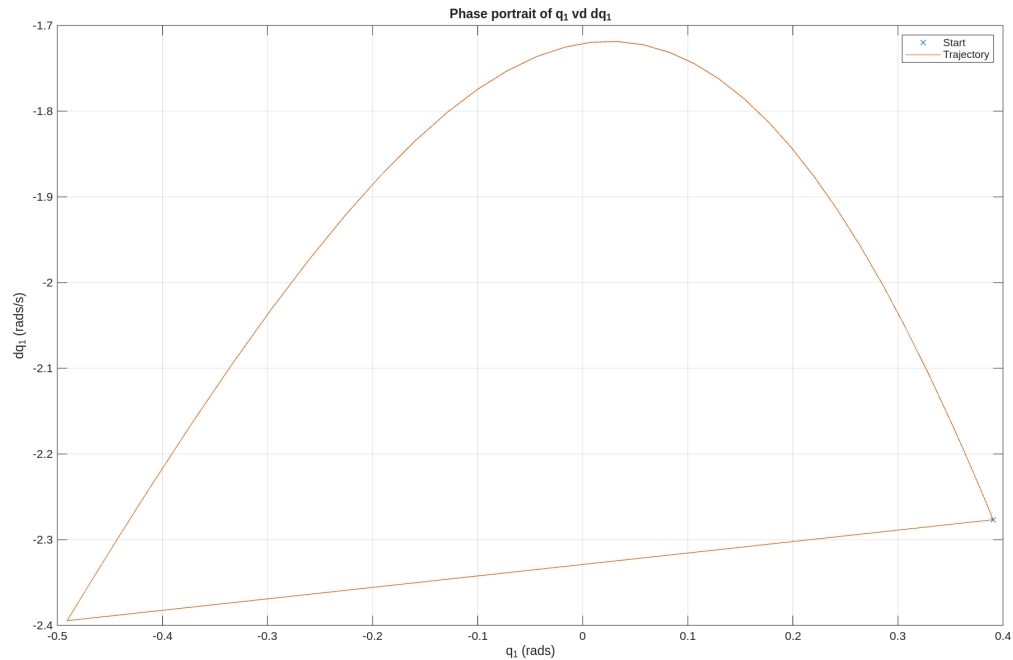


Figure 1: Phase portrait of the zero dynamics (q_1 vs. $\dot{q}_1$).